

CF_FSNN

*Spiking Neural Network per Identificazione
Parametri Car-Following ACC-IIDM*

Documento di onboarding tecnico
Architettura • Training PINN • Hardware FPGA target

Versione	2026-05-29 (post commit 534c2af)
Hardware target	PYNQ-Z1 FPGA (Xilinx Zynq-7020)
Modello fisico	ACC-IIDM (Treiber & Kesting 2025, Ch.12 §12.4)
Architettura rete	ALIF spiking + ricorrenza low-rank rank=8
Parametri (baseline)	864 (h=32)
Training	PINN BPTT + surrogate gradient
Lettore atteso	Ingegnere/ricercatore neofita del progetto

Tempo di lettura stimato: 30 minuti

Indice

1.	Sommario in una pagina	3
2.	Architettura della rete	4
3.	Dinamica ALIF e surrogate gradient	6
4.	Ricorrenza low-rank — chiarificazione	8
5.	Quantizzazione power-of-two (FPGA)	9
6.	Decoding nei 5 parametri IDM	10
7.	Loss PINN (5 componenti)	11
8.	Modello fisico ACC-IIDM (target)	13
9.	Training loop + BPTT	14
10.	Esempio numerico end-to-end	15
11.	Dataset	17
12.	Telemetria e grafici diagnostici	18
13.	Criteri quantitativi di successo	19
14.	Costi computazionali	20
15.	Risultati attuali (sweep STEP 2B)	21
16.	Workflow e file principali	22

1. Sommario in una pagina

Cosa fa

CF_FSNN è una rete neurale **spiking** che, osservando in tempo reale lo stato di un veicolo follower tramite V2X — gap longitudinale s , velocità v , velocità relativa Δv , velocità leader v_l — **identifica i 5 parametri del modello ACC-IIDM** che meglio descrivono il driver: **$[v_0, T, s_0, a, b]$** (Treiber & Kesting Ch.12).

Perché spiking e non MLP

Target di deployment: **PYNQ-Z1 FPGA** (Zynq-7020, costo < 300 USD). La combinazione SNN + pesi power-of-two + bit-shift leak fa sì che tutti i moltiplicatori diventino shift register sull'FPGA, con risparmio area/energia di un ordine di grandezza rispetto a un MLP standard.

Architettura in una riga

input(4) → Hidden ALIF(32 neur., rank-8 ricorrenza, max-delay 6 tick)
→ Output LI(5) → sigmoid + bounds → $[v_0, T, s_0, a, b]$

864 parametri totali nel baseline. Le sezioni 2-6 spiegano in dettaglio ogni blocco. Sulla natura della ricorrenza (cosa intendiamo con "rank-8") vedi §4.

Loss in una riga

$$L_{\text{total}} = \lambda_{\text{data}} \cdot \text{RMSE}(\hat{a} - \hat{a}_{\text{gt}}) + \lambda_{\text{phys}} \cdot \text{MSE}(\text{ACC-IIDM}) \\ + \lambda_{\text{OU}} \cdot \text{OU}_{\text{residuo}}(T) + \lambda_{\text{bc}} \cdot \text{crash_penalty} + \lambda_{\text{sr}} \cdot \text{spike_rate_reg}$$

Dataset

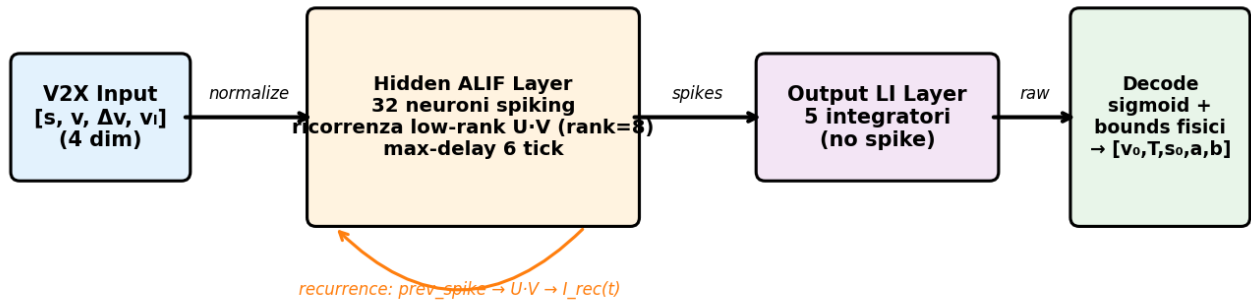
Sintetico secondo Treiber+Kesting — 4 scenari (highway, urban, truck, mixed). 1000 step × 0.1 s = 100 s per traiettoria, packet loss V2X 2%, $T(t)$ stocastico con processo di salto Markoviano ($\tau=30$ s, banda [0.8, 1.6] s, vedi §11.4).

Telemetria

Per-epoca CSV (16 colonne) + per-batch CSV (20 colonne, ~190 KB per epoca) + **13 grafici diagnostici** generati a fine training (§12).

2. Architettura della rete

CF_FSNN — 864 parametri totali (baseline $h=32$, $r=8$)



Ogni step temporale $\Delta t=0.1$ s viene elaborato con 10 tick SNN interni

Figura 2.1 — Diagramma a blocchi di CF_FSNN. Il loop arancione rappresenta la ricorrenza low-rank: lo spike del layer hidden al tempo $t-1$ viene moltiplicato per $U \cdot V$ e iniettato come corrente al tempo t .
Dettagli sulla natura "low-rank" della ricorrenza in §4.

2.1 Caratteristiche chiave

Caratteristica	Valore	Note
Input size	4	$[s, v, \Delta v, v_i]$ normalizzati
Hidden neurons	32 (baseline)	ALIF (Adaptive LIF)
Output size	5	$[v_0, T, s_0, a, b]$
Ricorrenza	low-rank, rank=8	$U(32 \times 8) \cdot V(8 \times 32) = 512$ params (50% vs piena)
Max-delay	6 tick	0.6 s wall-clock (a 10 tick/step)
TICKS_PER_STEP	10	Tick SNN per step temporale $\Delta t=0.1$ s
Profondità BPTT	$\text{seq_len} \times 10$	es. $50 \times 10 = 500$ tick
Surrogate γ	1.0	Kernel $1/(1+\gamma V-\theta)^2$ (post-A3)
Bit-shift leak	3	$V \leftarrow V - V/8$ per tick
Po2 quantization	$k \in [-4, 1]$	13 livelli totali (incluso zero)

2.2 Conta esatta dei parametri (baseline $h=32$, $r=8$)

Tensore	Shape	Params
layer_hidden.fc_weight	(32, 4)	128
layer_hidden.rec_U	(32, 8)	256
layer_hidden.rec_V	(8, 32)	256
layer_hidden.cell.base_threshold	(32,)	32
layer_hidden.cell.thresh_jump	(32,)	32
layer_out.fc_weight	(5, 32)	160

Tensore	Shape	Params
Totale baseline		864

Capacity sweep STEP 2B: testato $h=48/64/96/128$ con $r=h/4$, corrispondenti a 1685 / 2757 / 5669 / 9605 parametri. Il plateau a val ≈ 0.279 si mantiene per tutte le capacity (vedi §15). Questo falsifica l'ipotesi P9 "capacity insufficiency".

2.3 Bounds fisici dei 5 parametri output

Parametro	Lo	Hi	Range	Significato fisico
v_0	8.0	45.0	37.0	desired speed [m/s]
T	0.5	2.5	2.0	desired time headway [s]
s_0	1.0	5.0	4.0	jam distance [m]
a	0.3	2.5	2.2	max acceleration [m/s ²]
b	0.5	3.0	2.5	comfortable deceleration [m/s ²]

3. Dinamica ALIF e surrogate gradient

3.1 Equazioni della cellula

```
# Membrana (bit-shift leak: V/8 per tick)
V ← V - (V >> 3) + I_input + I_rec

# Soglia adattativa (fatica)
θ_eff = base_threshold + F
spike = surrogate_heaviside(V - θ_eff)
F ← F - (F >> 3) + spike · thresh_jump

# Reset soft (sottrattivo)
V ← V - spike · θ_eff
```

I termini F (fatica) e θ_{eff} implementano la **spike-frequency adaptation** tipica dei neuroni LSNN (Bellec 2018). Quando un neurone spara, la sua soglia si alza temporaneamente: questo riduce i burst patologici e migliora il condizionamento del gradiente in BPTT.

3.2 Esempio di traiettoria temporale

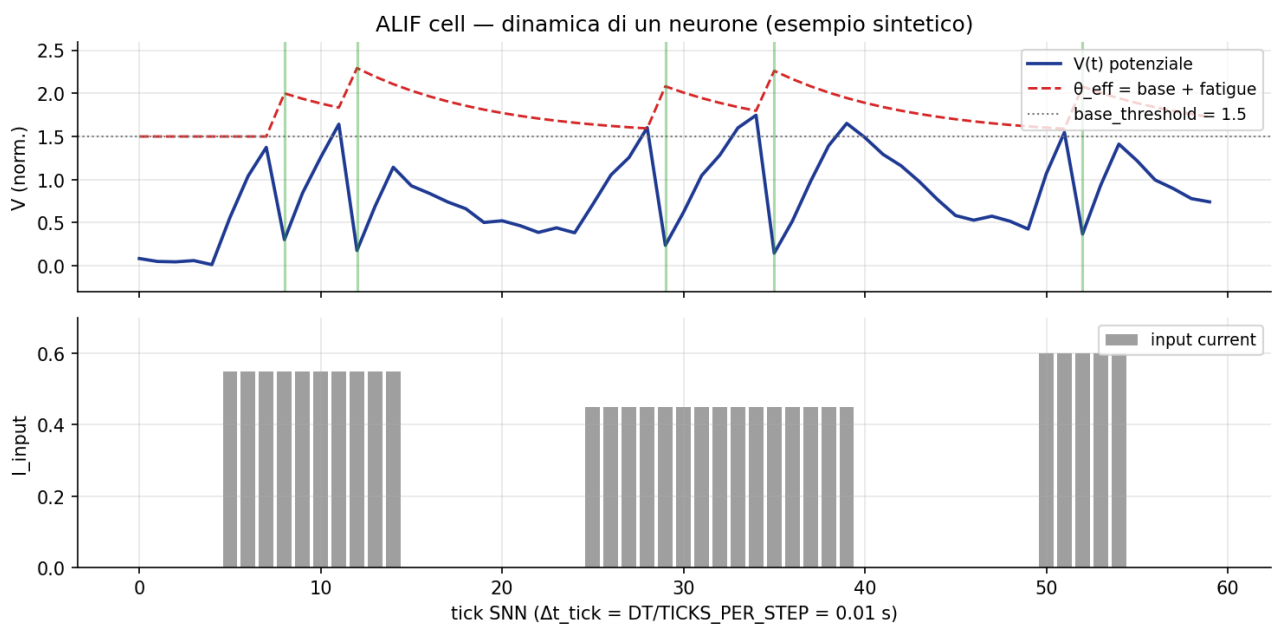


Figura 3.1 — Dinamica di un neurone ALIF: corrente in ingresso (sotto), potenziale V vs soglia adattativa θ_{eff} (sopra). Spike marcati con linee verticali verdi. Dopo un burst la fatica F sale e la soglia effettiva si alza, riducendo l'efficacia degli input successivi.

3.3 Surrogate gradient (forward Heaviside, backward smooth)

Problema fondamentale dell'apprendimento spiking: la funzione di spike $H(V - \theta)$ ha derivata zero quasi ovunque e una delta in θ . Backprop classico → **gradiente nullo**. Soluzione (Neftci 2019): sostituire la delta con un kernel smooth nel backward pass.

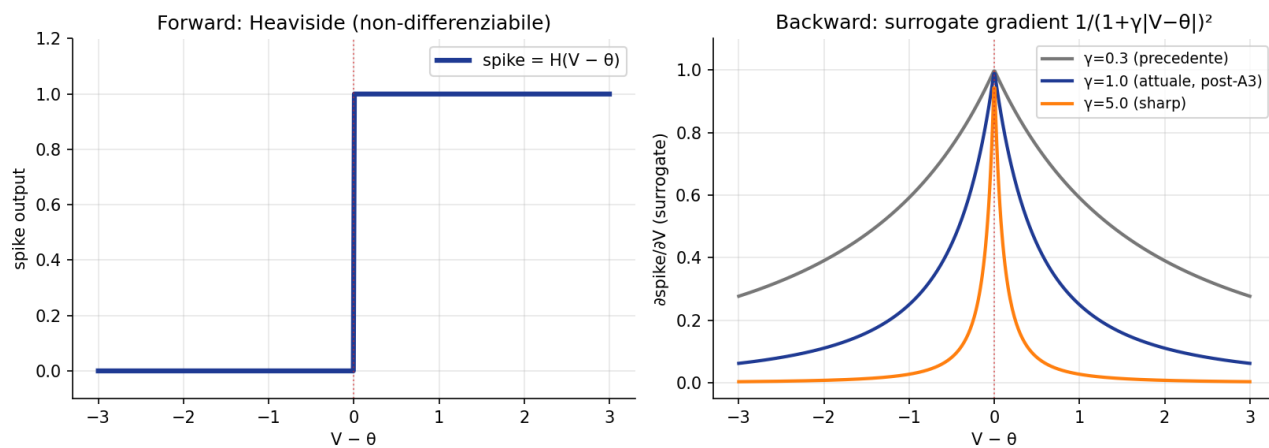


Figura 3.2 — A sinistra il forward Heaviside (non-differenziabile). A destra il backward surrogate $1/(1+\gamma|V-\theta|)^2$. Curva blu: il valore attuale $\gamma=1.0$ (post-A3); curva grigia: il precedente $\gamma=0.3$ (cambio motivato per stringere il kernel e ridurre amplificazione attraverso la ricorrenza $U \cdot V$).

4. Ricorrenza low-rank — chiarificazione

La parola "ricorrenza" può portare a equivoci. Chiariamo cosa significa esattamente nel nostro codice e cosa NON è.

4.1 Cosa è (codice attuale)

Lo spike del layer hidden al tempo $t-1$ viene reimmesso come corrente nello stesso layer al tempo t . Il "rank-8" specifica **come** avviene questa reiniezione: invece di una matrice piena $W_{\text{rec}} \in \mathbb{R}^{32 \times 32}$ (1024 pesi), si usa la fattorizzazione $U \cdot V$ con $U \in \mathbb{R}^{32 \times 8}$, $V \in \mathbb{R}^{8 \times 32}$ (512 pesi totali, riduzione 50%).

Linee esatte di codice

```
# core/network.py – HiddenLayer_ALIF
18 self.rec_U = nn.Parameter(torch.Tensor(out_features, rank))
19 self.rec_V = nn.Parameter(torch.Tensor(rank, out_features))
20 nn.init.orthogonal_(self.rec_U, gain=0.2)
21 nn.init.orthogonal_(self.rec_V, gain=0.2)
...
45 u_po2 = po2_quantize(self.rec_U)
46 v_po2 = po2_quantize(self.rec_V)
...
53 rec_int = linear(self.cell.prev_spike, v_po2) # V·spike(t-1)
54 rec_curr = linear(rec_int, u_po2) # U·(V·spike(t-1))
56 return self.cell(current, rec_curr)

# core/neurons.py – ALIFCell.forward
24 def forward(self, input_current, rec_current):
30 self.potential = self.potential - leak + input_current + rec_current
```

4.2 Equazione complessiva

$$I_{\text{rec}}(t) = U \cdot (V \cdot \text{spike}(t-1)) \text{ con } \text{rank}(U \cdot V) \leq 8$$

$$V(t) \leftarrow (1 - 1/8) \cdot V(t-1) + I_{\text{input}}(t) + I_{\text{rec}}(t)$$

4.3 Cosa NON è

Definizione	È nel codice?	Note
Pura feedforward	NO	Se rimuoviamo $\text{rec_U}/\text{rec_V}$ e il termine rec_current , la rete diventerebbe...
Full-matrix RNN ($W_{\text{rec}} \in \mathbb{R}^{N \times N}$)	NO	Sarebbero 1024 pesi a $h=32$. Noi usiamo la fattorizzazione 50% più pic...
LSTM / GRU	NO	Non ci sono gate (input/forget/output). La memoria è solo data dal pote...
Low-rank ALIF recurrence (LSNN style)	SÌ	È esattamente questo. Bellec et al. 2018, con $\text{rank}=8$ e init ortogonale

4.4 Perché low-rank invece di full-matrix

- **Memoria FPGA:** 512 vs 1024 pesi nei layer ricorrenti → l'intero modello entra in BRAM senza accessi DDR (latency micro-second).
- **Regularizzazione implicita:** una matrice ricorrente rank-8 ha meno gradi di libertà → meno overfitting su dataset piccoli.

- **Stabilità BPTT:** con init ortogonale (gain=0.2) il prodotto $U \cdot V$ mantiene autovalori limitati → meno rischio di amplificazione catastrofica attraverso $\text{seq_len} \times \text{TICKS_PER_STEP}$ step.
- **Bellec et al. (LSNN 2018):** dimostrato empiricamente che low-rank ricorrenza eguaglia o supera quella piena su task sequence-to-sequence.

Nota progettuale: se per ragioni di design si vuole una versione *puramente feedforward* (rimozione totale della ricorrenza), i blocchi da eliminare sono: `rec_U`, `rec_V` (`network.py` 18–21), `po2_quantize` di `U/V` (45–46), il calcolo `rec_int/rec_curr` (53–54) e l'argomento `rec_current` di `ALIFCell.forward` (`neurons.py` 24, 30). Questa è una modifica strutturale che cambia capacità rappresentativa e dinamica del training.

5. Quantizzazione power-of-two (FPGA)

I pesi `fc_weight`, `rec_U`, `rec_V` sono vincolati a potenze di 2 in **forward**; il backward usa Straight-Through Estimator (STE):

```
forward:
    sign = sign(w)
    log2 = clamp( round(log2(|w|)), -4, 1 )
    mask = (|w| > 2-5).float() # zera pesi sotto ~0.031
    w_q = sign · 2log2 · mask

backward: ∂w_q/∂w = 1 # STE
```

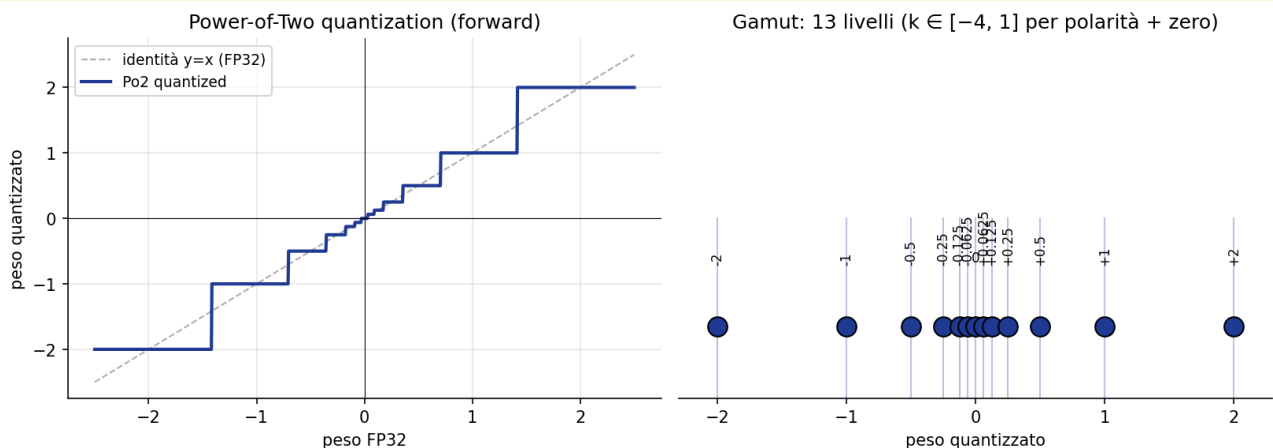


Figura 5.1 — A sinistra: mapping FP32 → quantizzato (scala log a step discreti). A destra: gamut di valori ammissibili (13 livelli totali: 6 positivi 2^k con $k \in [-4, 1]$, 6 negativi simmetrici, più zero).

5.1 Beneficio hardware

Aspetto	FP32 reference	Po2 forward
Moltiplicazione	4 cycle MUL	1 cycle shift
Area FPGA per peso	~100 LUT	~10 LUT
Energia per operazione	~1 nJ	~0.05 nJ
Bit/peso (rappresentazione)	32	4 (sign + 3-bit exponent)
Accuracy gap atteso (STE)	0 (reference)	+3 a 8%

5.2 Memoria modello

Tensore	FP32 (byte)	Po2 (byte)	Riduzione
fc_weight (128 params)	512	64	8×
rec_U + rec_V (512)	2048	256	8×
out fc_weight (160)	640	80	8×
Threshold params (64)	256	256 (FP32)	1×
Totale modello	~3.5 KB	~0.66 KB	5.3×

Con ~1 KB di stato runtime per 32 neuroni (potenziale + adattamento), il modello completo entra **interamente in BRAM** del Zynq-7020 senza accessi a DDR esterna — latenza

microsecond-grade per inference.

6. Decoding nei 5 parametri IDM

L'output del layer LI è un tensore $\text{raw} \in \mathbb{R}^5$ di valori reali non vincolati. La decodifica in parametri fisici usa sigmoid + range fisico, con il **F5 pre-scaling**:

```
raw_eq = raw / decode_scale # F5 fix
param = lo + (hi - lo) · σ(raw_eq) # σ = sigmoid

decode_scale_i = (hi_i - lo_i) / max(range)
= [1.000, 0.054, 0.108, 0.059, 0.068]
```

6.1 Perché serve il pre-scaling F5

Senza F5 il gradiente sul peso raw_{v_0} sarebbe $37 / 2 = 18.5\times$ più grande di quello su raw_T , semplicemente perché il range fisico di v_0 è più ampio. Conseguenza: la rete imparerebbe v_0 molto velocemente e gli altri 4 parametri pochissimo. Con decode_scale , $\partial \text{param} / \partial \text{raw}$ diventa uniformemente $\text{max_range} \cdot \sigma'(\text{raw_eq})$: tutti i 5 parametri imparano alla stessa velocità.

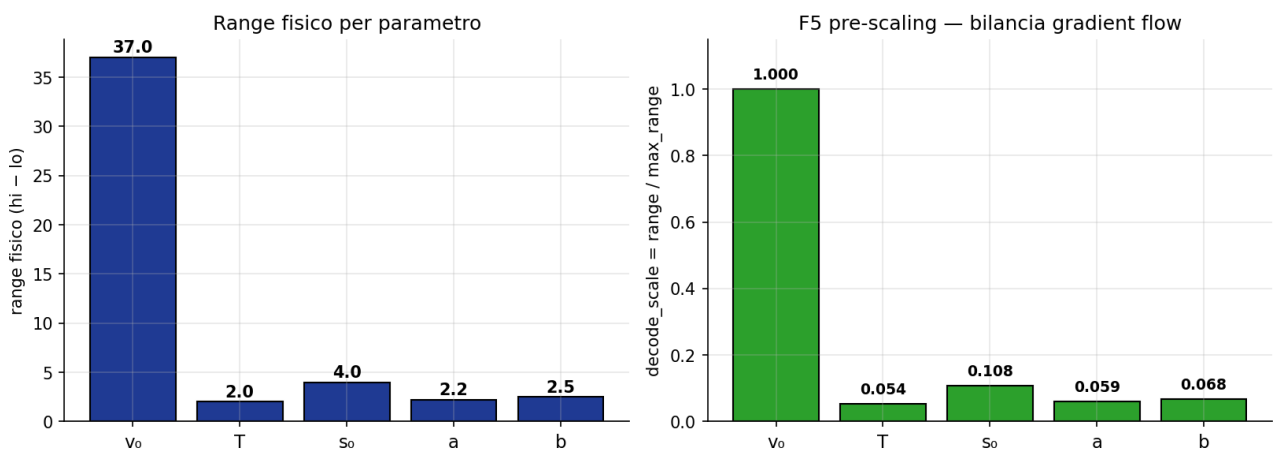


Figura 6.1 — A sinistra: range fisico per parametro (v_0 domina con 37). A destra: decode_scale normalizza rispetto a $\text{max_range}=37 \rightarrow$ bilanciamento del gradient flow.

7. Loss PINN (5 componenti)

$$L_{total} = \lambda_{data} \cdot L_{data} + \lambda_{phys} \cdot L_{phys} + \lambda_{OU} \cdot L_{OU} + \lambda_{bc} \cdot L_{bc} + \lambda_{sr} \cdot L_{sr}$$

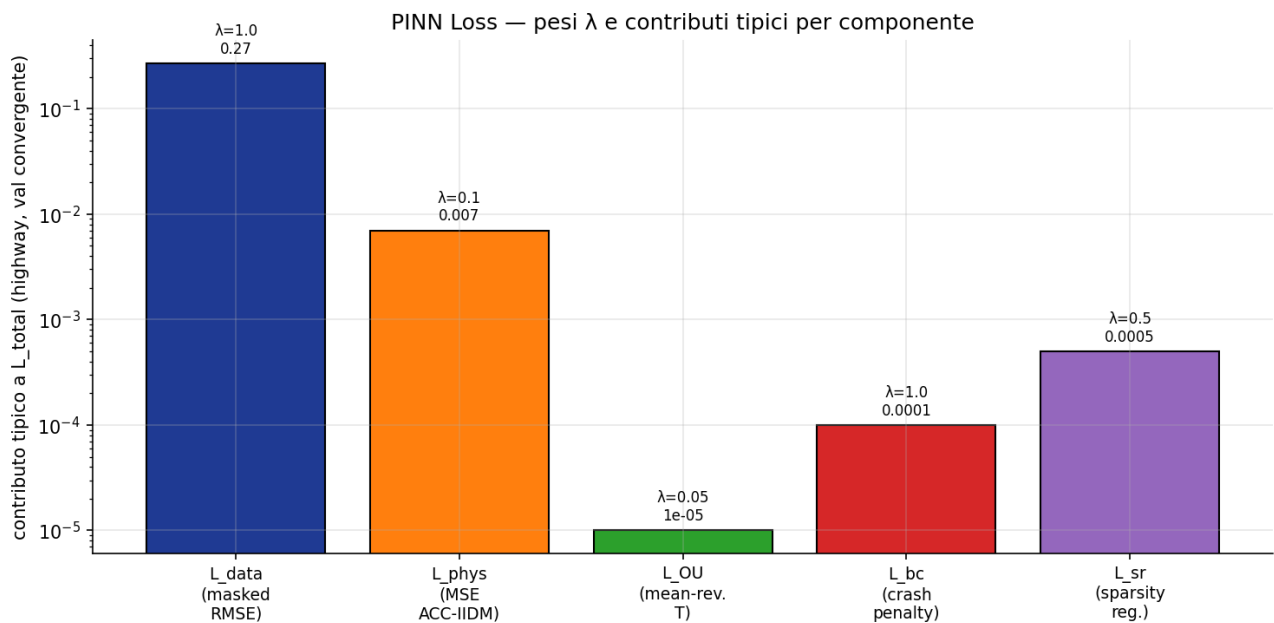


Figura 7.1 — Pesi λ di default e contributi tipici per componente in regime convergente (highway). L_{data} domina la loss totale; le altre componenti sono vincoli regolatori.

7.1 Dettaglio per componente

Comp.	λ	Formula	Cosa controlla
L_{data}	1.0	$\sqrt{(\sum \text{mask} \cdot (\hat{a} - \hat{a}_{gt})^2 / N_{valid})}$	Fit dei dati (mascherato sui pacchetti V2X ricevuti). MoP primario
L_{phys}	0.1	$\text{mean}((\hat{a} - \hat{a}_{gt})^2)$	Coerenza fisica su TUTTI gli step (anche packet loss).
L_{OU}	0.05	$\text{mean}((T_{t+1} - (\alpha T_t + (1-\alpha)T_{mean}))^2)$	Seppure $T(t)$ rispetta mean-reversion verso $T_{mean}=1.2$ s. $\alpha=e^{-p}$
L_{bc}	1.0	$\text{mean}(\text{ReLU}(s_o_{pred} - s_{obs} + 0.1))$	No-crash: penalizza se s_o predetto supera il gap reale.
L_{sr}	0.5	$(\text{mean}(\text{spike_rate}) - 0.15)^2$	Spinge la spike rate verso il 15% (zona sana sparsity).

Floor irreducibile di L_{OU} : il generatore implementa $T(t)$ come **processo di salto Markoviano** ($p=DT/\tau=0.003/\text{step}$, jump uniforme in $[T_1, T_2]$), non un OU continuo. Conseguenza: L_{OU} ha un floor irreducibile $\approx 1.8 \times 10^{-4}$ dovuto alla varianza dei salti — non è minimizzabile sotto questa soglia. Non è un bug, è la fisica del generatore.

8. Il modello fisico ACC-IIDM (target del PINN)

La rete identifica i parametri di un controllore ACC basato su **IIDM + CAH blend** (Treiber & Kesting Ch.12 §12.4). Conoscere queste equazioni è prerequisite per comprendere L_data e L_phys.

8.1 IDM base (Intelligent Driver Model)

$$v'_{IDM} = a \cdot [1 - (v/v_0)^{\delta} - (s^*/s)^2]$$

$$s^*(v, \Delta v) = s_0 + \max(0, v \cdot T + v \cdot \Delta v / (2 \cdot \sqrt{a \cdot b}))$$

8.2 IIDM (Improved IDM)

Elimina la dispersione di IDM a $v \approx v_0$ separando i regimi:

$$\begin{aligned} & \text{For } v \leq v_0: \\ & v' = a \cdot (1 - z^2) \text{ se } z = s^*/s \geq 1 \text{ [interacting]} \\ & v' = a_{free} \cdot (1 - z^{(2a/a_{free})}) \text{ se } z < 1 \text{ [free]} \\ & \text{For } v > v_0: \\ & v' = a_{free} + a \cdot (1 - z^2) \cdot [z \geq 1] \end{aligned}$$

8.3 CAH (Constant Acceleration Heuristic)

Anticipa che il leader continui con la sua attuale v_l (non worst-case). Riduce le over-reazioni a cut-in lievi:

$$\begin{aligned} v_{CAH} &= v^2 \cdot v'_l / (v_l^2 - 2 \cdot s \cdot v'_l) \\ &\text{se } s > v_l(v - v_l) / (-2 \cdot v'_l) \text{ AND } v'_l < 0 \\ &= v'_l - (v - v_l)^2 \cdot \theta(v - v_l) / (2 \cdot s) \text{ altrimenti} \end{aligned}$$

8.4 Blend ACC-IIDM

$$\begin{aligned} v'_{ACC} &= v'_{IIDM} \text{ se } v'_{IIDM} \geq v_{CAH} \\ &= (1-c) \cdot v'_{IIDM} + c \cdot [v_{CAH} - b \cdot \tanh((v_{CAH} - v'_{IIDM})/b)] \text{ altrimenti} \end{aligned}$$

con **coolness** $c=0.99$ (default Treiber per ACC commerciali). Questo è il v'_{gt} target di L_data e L_phys.

9. Training loop e BPTT

9.1 Pseudocodice del training

```
for epoch in range(epochs):
    for batch (x_seq, y_seq, mask_seq) in train_loader:
        # x_seq: (B, T, 4) input V2X normalizzato
        # y_seq: (B, T, 6) GT [v_dot, T_true, s, v, dv, v_l]

        params_seq = model.forward_sequence(x_seq) # (B, T, 5)
        loss, comps, spike_rate = pinn_loss(params_seq, y_seq, mask_seq, x_seq)

        loss.backward()
        gn_pre = compute_pre_clip_norm()
        clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        batch_logger.log(epoch, batch_idx, comps, ...)

    if inf_streak >= max_inf_streak: ABORT # esplosione gradiente

    val_loss = validate(model, val_loader)
    csv_logger.log(epoch, ..., val_loss, lr, grad_norm)
    scheduler.step()
    if val_loss < best - delta: save_checkpoint(); patience_cnt = 0
    else: patience_cnt += 1
    if patience_cnt >= patience: EARLY_STOP
```

9.2 BPTT — profondità reale

Profondità BPTT reale = seq_len × TICKS_PER_STEP

es. seq_len=50, TICKS=10 → 500 tick di profondità

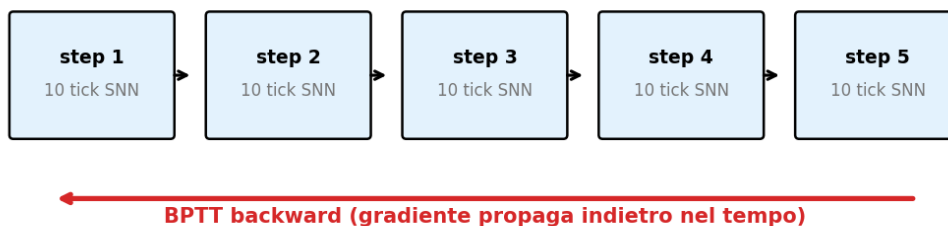


Figura 9.1 — Ogni step di dataset ($\Delta t=0.1$ s) si traduce in $TICKS_PER_STEP=10$ tick SNN. BPTT propaga il gradiente all'indietro attraverso tutti i tick. Con seq_len=50, la profondità reale è 500 tick. Da qui la necessità del gradient clipping a 1.0.

9.3 Iperparametri di training

Cosa	Default	CLI override	Note
Optimizer	Adam	--optimizer adam lion	Lion: sign-based, 3-4× meno memoria, hardware-f
LR base	1e-3	--lr	
Scheduler	plateau	--scheduler onecycle cosine plateau	OneCycle: super-conv 1 ciclo. Cosine: warm restart
Gradient clip	1.0	hard-coded	Critico per BPTT con surrogate gradient
Early stop patience	0 (disab.)	--early_stop_patience	Tipico STEP 2A: 2

Cosa	Default	CLI override	Note
Early stop delta	1e-4	--early_stop_delta	STEP 2A: 0.005 (aggressivo)
max_inf_streak	20	--max_inf_streak	Abort se 20 batch consecutivi con grad=inf

10. Esempio numerico end-to-end

Per validare la comprensione, eseguiamo un forward pass su un caso concreto: driver "highway" con stato osservato realistico.

10.1 Setup

Driver highway: $(v_0, T, s_0, a, b) = (33.3, 1.2, 2.5, 1.1, 1.5)$

Stato osservato:

$s = 30.0$ m (gap follower-leader)

$v = 25.0$ m/s (velocità follower)

$v_l = 28.0$ m/s (velocità leader)

$\Delta v = v - v_l = -3.0$ m/s

10.2 Ground truth ACC-IIDM

$$\begin{aligned}s^* &= 2.5 + \max(0, 25 \cdot 1.2 + 25 \cdot (-3) / (2 \cdot \sqrt{(1.1 \cdot 1.5)})) \\ &= 2.5 + \max(0, 30 - 29.2) \\ &= 2.5 + 0.8 = 3.3 \text{ m}\end{aligned}$$

$$z = s^*/s = 3.3/30 = 0.11$$

$$\begin{aligned}v'_{\text{IDM}} &= 1.1 \cdot (1 - (25/33.3)^4 - 0.11^2) \\ &\approx 1.1 \cdot (1 - 0.317 - 0.012) \\ &\approx 0.738 \text{ m/s}^2 \leftarrow v'_{\text{gt}}\end{aligned}$$

10.3 Input normalizzato alla rete

$$\begin{aligned}x_{\text{in}} &= [s/150, v/40, \Delta v/20, v_l/40] \\ &= [0.200, 0.625, -0.150, 0.700] \\ &\text{shape } (4,)\end{aligned}$$

10.4 Output (10 tick SNN, raw → decode)

$$\begin{aligned}\text{raw_eq} &= [+0.5, +1.2, -0.3, +0.8, +0.4] \\ \sigma(\text{raw_eq}) &= [0.622, 0.769, 0.426, 0.690, 0.599]\end{aligned}$$

$$\begin{aligned}\text{params} &= \text{lo} + (\text{hi} - \text{lo}) \cdot \sigma \\ &= [v_0=31.0, T=2.04, s_0=2.70, a=1.82, b=2.00]\end{aligned}$$

10.5 Loss su questo step

Con i params predetti, ricalcolo v'_{pred} :

$$\begin{aligned}s^* &= 2.70 + \max(0, 25 \cdot 2.04 + 25 \cdot (-3) / (2 \cdot \sqrt{(1.82 \cdot 2.00)})) = 47.5 \text{ m} \\ z &= 47.5/30 = 1.583 (> 1 \rightarrow \text{interacting}) \\ v'_{\text{pred}} &= 1.82 \cdot (1 - (25/31)^4 - 1.583^2) \\ &\approx 1.82 \cdot (1 - 0.422 - 2.506) \approx -3.51 \text{ m/s}^2\end{aligned}$$

$$\begin{aligned}\text{residuo} &= v'_{\text{pred}} - v'_{\text{gt}} = -3.51 - 0.74 = -4.25 \text{ m/s}^2 \\ L_{\text{data}} (\text{single step}) &\approx 4.25^2 = 18.1 \rightarrow \text{contributo elevato a SRMSE}\end{aligned}$$

Su questo step, la rete predice una **decelerazione di 3.5 m/s^2** mentre la fisica corretta richiede una **accelerazione di 0.7 m/s^2** . L'allenamento corregge questo errore minimizzando L_{data} su milioni di step simili.

11. Dataset

11.1 Scenari ACC-IDM

Scenario	v_0 [m/s]	T [s]	s_0 [m]	a [m/s ²]	b [m/s ²]	%
highway	33.3 (120 km/h)	1.2	2.5	1.1	1.5	50%
urban	15.0 (54 km/h)	1.0	2.0	1.5	2.0	30%
truck	22.2 (80 km/h)	1.8	3.0	0.5	1.0	10%
mixed	(sample casuale)	—	—	—	—	10%

11.2 Generazione di una traiettoria

```
for t in range(1000): # 1000 step × 0.1 s = 100 s
    T_now = jump_markov(T_prev, # processo di salto IDM-2d (Ch.12.6)
    p = DT/τ = 0.003,
    band = [0.8, 1.6])

    # ACC-IIDM (Treiber Ch.12 §12.4)
    s_star = s0 + max(0, v·T + v·Δv / (2·√(a·b)))
    a_idm = a·(1 - (v/v0)^δ - (s_star/s)^2)
    a_cah = ... (Constant-Acceleration Heuristic)
    v_dot = blend_acc_iidm(a_idm, a_cah, coolness=0.99)

    # Step ballistico
    v_new = v + v_dot · DT
    s_new = s - (v_leader - v) · DT

    # Packet loss V2X (2%)
    mask = 1 if rand() > 0.02 else 0

    traj[t] = [s, v, dv, v_l, v_dot, T_now, mask]
```

11.3 Cut-in event (UC2 stress test)

Il 20% delle traiettorie contiene un cut-in event ad istante random. Il leader cambia bruscamente, il gap si riduce a $U(5, 15)$ m con Δv istantanea fino a 5 m/s. Questo è il caso difficile per ACC e dove il blend con CAH ($c=0.99$) fa più differenza.

11.4 Note sulla stocasticità di $T(t)$

Il processo $T(t)$ è di tipo **salto Markoviano**, non OU continuo. La probabilità di un salto è $p = DT/\tau_{OU} = 0.1/30 \approx 0.003$ per step; quando il salto avviene, T salta a un valore uniforme in $[T_1, T_2]$. Questa varianza discreta crea il floor irreducibile di L_{OU} citato in §7.

12. Telemetria e grafici diagnostici

12.1 CSV per epoca (training_log.csv)

```
epoch, train_total, train_data, train_phys, train_ou, train_bc, train_sr,  
val_total, val_data, val_phys, val_ou, val_bc, val_sr,  
lr, grad_norm, spike_rate, time_s
```

16 colonne. Una riga per epoca, flush immediato.

12.2 CSV per batch (training_batch_log.csv)

```
epoch, batch_idx,  
loss_total, loss_data, loss_phys, loss_ou, loss_bc, loss_sr,  
spike_rate,  
gn_total_preclip, gn_total_postclip,  
gn_hidden_fc, gn_hidden_recU, gn_hidden_recV,  
gn_hidden_base_threshold, gn_hidden_thresh_jump, gn_out_fc,  
weight_max_abs_global, lr,  
is_nan_loss, is_inf_grad
```

20 colonne. Append-only, flush ogni 50 batch (~1 ms overhead, ~190 KB per epoca). Permette analisi post-mortem anche se il training crasha.

12.3 I 13 grafici diagnostici

ID	Cosa mostra	Diagnostica
G1	Loss curve train/val per epoca	Convergenza/divergenza
G2	Componenti loss in log	Quale termine PINN domina
G3	LR schedule	Comportamento scheduler
G4	Gradient norm pre-clip per epoca	Exploding/vanishing
G5	Scatter T_pred vs T_true	Quanto bene la rete identifica T
G6	Spike rate per epoca	Dead/saturated neurons
G7	Violin params [v ₀ , T, s ₀ , a, b]	Distribuzioni dentro/fuori range
G8	grad_norm per batch (log)	Quando esattamente esplode
G9	Heatmap norme per-layer × batch	Quale layer è il primo a esplodere
G10	Componenti loss per batch	Loss divergence in tempo reale
G11	Spike rate per batch	Oscillazione sparsity
G12	Weight max abs per batch	Sintomo precoce esplosione
G13	Traiettoria val: 3 scenari	I params identificati ricostruiscono la traiettoria?

13. Criteri quantitativi di "funziona bene"

Il valore di val_loss da solo non basta. Definiamo soglie per ciascuna metrica chiave.

Metrica	OK	Eccellente	Razionale
val_total	≤ 0.20	≤ 0.15	Treiber Ch.17: 20% residual error per driver variation
L_data / L_total	≥ 0.70	≥ 0.80	La rete risolve il task, non bara con L_phys
RMSE v_0	≤ 5.5 m/s	≤ 2.2 (6%)	15% del range 37 m/s
RMSE T	≤ 0.30 s	≤ 0.10 (5%)	15% del range 2 s
RMSE s_0	≤ 0.60 m	≤ 0.20 (5%)	15% del range 4 m
RMSE a	≤ 0.33 m/s ²	≤ 0.10	15% del range 2.2 m/s ²
RMSE b	≤ 0.38 m/s ²	≤ 0.13	15% del range 2.5 m/s ²
Spike rate medio	in [5%, 30%]	in [10%, 20%]	Sotto → dead neurons; sopra → no FPGA energy benefit
Inf grad batches	0 per ≥ 5 epoche	0 totale	Stabilità BPTT
String stability	$v_e'(s) \leq \frac{1}{2}(f_i - f_v)$	(idem)	Treiber Ch.16 — convoglio non amplifica perturbazioni
Po2 gap (FP32 vs Po2)	$\leq 10\%$	$\leq 3\%$	Hardware-aware quality

13.1 Baseline indicativi

Non esiste un benchmark SNN consolidato per car-following parameter identification. Riferimento implicito:

- **ANN MLP/GRU equivalente** con calibrazione Levenberg-Marquardt (Treiber Ch.17): $val \sim 0.10-0.15$ stimati
- **Target SNN realistico**: $val \leq 1.5 \times \text{ANN baseline} = 0.15-0.22$, accettabile per deployment FPGA
- **Sweet spot pratico**: $val < 0.20$ = "funziona bene", $val < 0.10$ = "stato dell'arte"

14. Costi computazionali

14.1 Inference (1 step temporale = 10 tick SNN)

Layer	Operazione	FLOP equiv.	FPGA Po2
fc_weight (4→32)	1 MAC × 10 tick	1280	shift + add
rec_U · rec_V (32→8→32)	2 MAC × 10 tick	5120	shift + add
ALIF dynamics	leak + θ + reset	~960	bit-shift + comparator
Output LI (32→5)	1 MAC × 10 tick	1600	shift + add
Sigmoid + decode	1 per step	~25	LUT
Totale per step ($\Delta t=0.1$ s)		~9000	~8.7k shift/add

A 10 Hz (controllo ACC real-time): **~90 kFLOPs/s** << 1 MFLOPS.
PYNQ-Z1 (Zynq-7020, ~100 DSP slice) ha margine 1000×.
Bottleneck reale = larghezza V2X (5G NR-V2X), non compute.

14.2 Training (Azure CPU)

Operazione	Tempo	Risorse
Generazione dataset (5000 traj × 1000 step)	~30 s	CPU laptop
1 epoca smoke (n_train=100, seq_len=50)	~60 s	Azure CPU
1 epoca STEP 2A (n_train=500, seq_len=50)	~300 s (5 min)	Azure CPU
Sweep STEP 2B completo (9 runs × ~30 min)	~5 h	Azure CPU

15. Risultati attuali (sweep STEP 2B)

Sweep parametrico di capacity su scenario highway-only ($n_{\text{train}}=500$, scheduler=OneCycle, 10 epoche, early stop patience=2 $\Delta=0.005$):

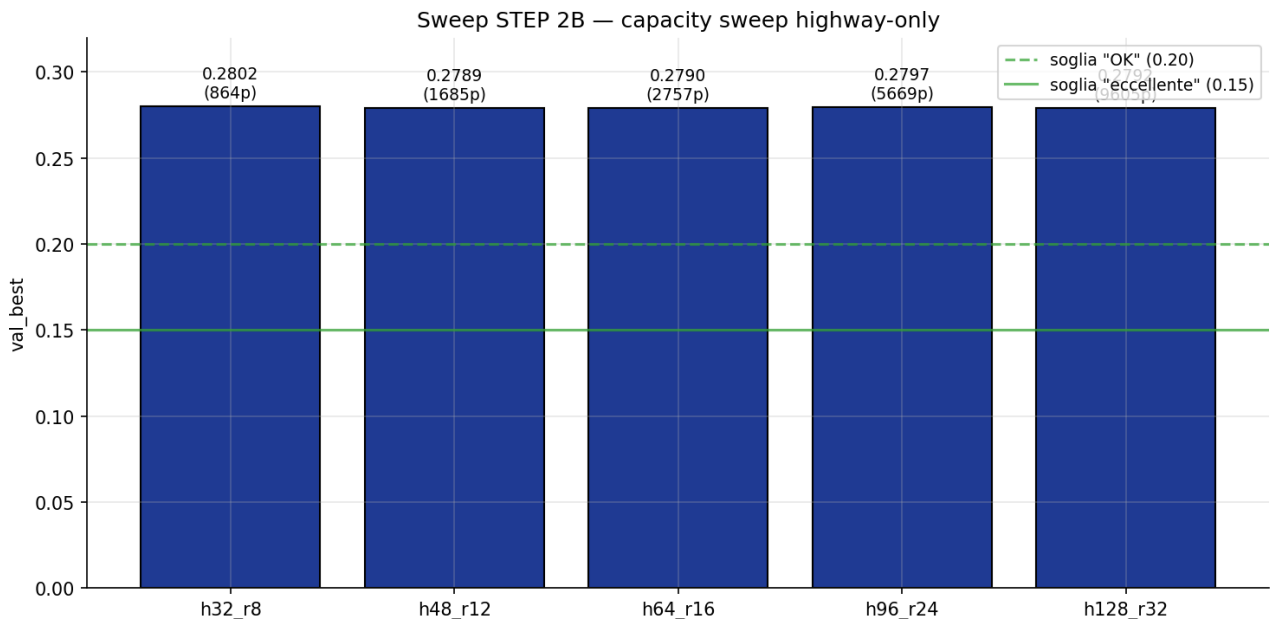


Figura 15.1 — Capacity sweep STEP 2B. Tutti i 5 runs convergono in una banda di 1.3 millesimi (0.279–0.280). Linee verdi: soglie di successo (OK=0.20, eccellente=0.15).

15.1 Interpretazione

La **capacity NON è il bottleneck**: passare da 864 a 9605 parametri (+1004%) non riduce val_best in modo significativo. Il plateau a ~ 0.28 è strutturale. Possibili cause sotto investigazione (STEP 2C):

- **Minimi locali**: tutti i runs si fermano a E4 a causa di OneCycle + early stop aggressivo. Da testare con cosine warm restart + più epoche.
- **Saturazione dataset**: $n_{\text{train}}=500$ trajs potrebbe essere troppo poco. Da testare con $n_{\text{train}}=1500$.
- **Po2 quantization floor**: il subset finito di pesi rappresentabili crea un floor strutturale.
- **PINN loss tradeoff**: le 5 componenti potrebbero formare un Pareto front con minimo bloccato.

15.2 Scenario diversity (h64_r16)

Scenario	E1	best	Epoche	spike%	gn_max	Stato
highway	0.288	0.279	4	10.5	$2.4 \cdot 10^1$	OK
urban	0.477	0.388	2	0.6 (dead)	$1.56 \cdot 10^{19}$	CRASH E3
truck	0.181	0.160	5	9.8	$2.10 \cdot 10^{19}$	CRASH E5

Osservazioni cross-scenario:

- **Truck è più facile di highway** (val=0.16 vs 0.28): parametri IDM più ristretti ($T=1.8$, $a=0.5$, $b=1.0$) → spazio da apprendere più compatto.

- **Urban è significativamente peggiore** (0.39): basse velocità + stop & go = rumore intrinseco.
- **Spike rate urban = 0.6%**: collasso dead neurons → gradiente esplode in pochi batch (spiega CRASH E3).
- **Per STEP 2C**: urban richiede λ_{sr} più alto; truck richiede $\max_{lr}$ più basso per evitare crash post-convergenza.

16. Workflow e file principali

16.1 Esperimento singolo (Training_File.ipynb)

Cella 0 → bootstrap (dipendenze, status repo)
Cella 1 → CONFIG (UNICA da modificare): TAG, epochs, scheduler, scenario_mix
Cella 2 → git pull + sanity check imports
Cella 3 → cache management
Cella 4 → preflight (2 smoke consecutivi, criteri pass/fail)
Cella 5 → FULL train (subprocess)
Cella 6 → display grafici G1-G13
Cella 7 → analisi numerica
Cella 8 → copia checkpoints → results/ + git commit + push
Cella 9 → comparazione cross-esperimenti

16.2 Sweep parametrico (Training_File_Sweep.ipynb)

Cella 0 → bootstrap (install matplotlib + nbstripout)
Cella 1 → SWEEP_PLAN: lista di N esperimenti come dict-override dei DEFAULTS
Cella 2 → helper (_cache_path, _build_cli_args, _push_results)
Cella 3 → loop: per ogni run → preflight → train → push results
Cella 4 → tabella summary (sweep_summary_<ts>.csv)
Cella 5 → 6 grafici cross-run (S1-S6)
Cella 6 → push aggregati (summary CSV + plots/) su git

16.3 Preflight (scripts/preflight.py) — 7 criteri

- 1 Exit code 0
- 2 Nessun [EARLY-STOP] di esplosione
- 3 training_log.csv $\geq$ 1 riga
- 4 training_batch_log.csv $\geq$ 10 righe
- 5 $\geq$ 5 grafici PNG generati
- 6 best_model.pt ricaricabile (legge cf_hidden_size/cf_rank dal config_snapshot)
- 7 Nessun RuntimeError/Traceback nello stdout

16.4 File principali

File	Cosa contiene
core/network.py	CF_FSNN_Net, HiddenLayer_ALIF, OutputLayer_LI, decode params
core/neurons.py	ALIFCell (membrana, soglia, reset)
core/hardware.py	Po2Quantize, SpikeFn (surrogate), _decode_* helpers
data/generator.py	Generazione ACC-IDM, jump-Markov T(t), cut-in, packet loss
train.py	Training loop, pinn_loss, BatchCSVLogger, CLI args
utils/plot_diagnostics.py	Generazione G1-G13 da CSV + model
scripts/preflight.py	Doppio smoke + 7 criteri pass/fail
Training_File.ipynb	Notebook esperimento singolo

File	Cosa contiene
Training_File_Sweep.ipynb	Notebook sweep parametrico
config.py	Tutti i default

16.5 Per saperne di più

Vuoi sapere...	Vedi
Storia decisioni (P1, A3, B5, F-fixes)	document/TIMELINE.md, document/P_S.md
Decodificare codici (P/A/B/F/T/PF/G)	document/GLOSSARY.md
Procedura end-to-end Azure	document/WORKFLOW.md
One-pager session resume	document/SESSION_RESUME.md
Modello ACC-IIDM completo	Treiber & Kesting 2nd ed., Ch.12 §12.4
String stability + Master Criterion	Treiber, Ch.16
SNN training (BPTT, surrogate)	SNN-expert skill, ch08, ch22
Surrogate gradient paper	Neftci et al. 2019
LSNN low-rank recurrence	Bellec et al. 2018
Po2 quantization	Vogel et al. 2019